

CAHIER DES CHARGES

CAHIER DES CHARGES : Codexia Application d'apprentissage python

1. Objectif du Projet

Créer une application web interactive permettant aux utilisateurs d'apprendre et de s'entraîner en Python. L'application doit évaluer automatiquement le code soumis, attribuer des points d'expérience (XP) et débloquent des niveaux de difficulté progressifs selon un système de rangs.

2. Spécifications Fonctionnelles (Ce que fait l'application)

Pour l'utilisateur (Front-office) :

- **Système de compte** : Inscription, connexion, déconnexion.
- **Tableau de bord (Dashboard)** : Affichage du pseudo, du rang actuel, de la barre d'XP et des statistiques (nombre d'exercices résolus).
- **Catalogue d'exercices** : Liste des 50 exercices divisés par paliers. Les paliers supérieurs sont verrouillés tant que le rang requis n'est pas atteint.
- **Interface de résolution** : * Un espace pour lire l'énoncé.
 - Un éditeur de code intégré (avec coloration syntaxique).
 - Un bouton "Soumettre".
 - Une console d'affichage des résultats (Succès, Échec, ou Message d'erreur détaillé renvoyé par Python).

Pour la mécanique de jeu (Gamification) :

- Validation d'un exercice inédit = Gain d'XP.
- Passage automatique au rang supérieur lorsque le seuil d'XP est atteint.
- Déblocage automatique des exercices du palier correspondant au nouveau rang.

Pour la correction automatique (Logique métier) :

- Concaténation (fusion) invisible du code de l'utilisateur avec un script de tests (asserts) écrit par le créateur de l'exercice.
- Envoi du fichier complet à l'API Piston.
- Analyse de la réponse de Piston (Si la sortie standard ne contient pas d'erreurs, l'exercice est validé).

3. Spécifications Techniques (La Stack)

- **Front-end** : HTML, CSS, JavaScript.
 - *Outil recommandé* : Intégration de **Monaco Editor** ou **CodeMirror** en JS. (Ne fais pas coder tes utilisateurs dans un simple `<textarea>` HTML, c'est illisible et sans indentation).
- **Back-end** : PHP (Vanilla ou framework léger). Gère l'authentification, les requêtes à la base de données, et agit comme proxy vers Piston.
- **Moteur d'exécution** : API Piston (emkc.org pour démarrer).
- **Base de données** : MySQL ou SQLite.

- **Hébergement (cible)** : Serveur classique (VPS ou mutualisé type Hostinger/OVH) supportant PHP et requêtes cURL.

4. Structure des Rangs et Progression

Rang	Exos débloqués	Seuil d'XP requis	Explication de la tolérance (Exos sautés)
Débutant	1 à 15	0 XP	Acquis dès le départ.
Intermédiaire	16 à 25	1 300 XP	Le max possible est 1500 XP. L'utilisateur peut zapper 2 exos Débutant et passer quand même.
Avancé	26 à 34	2 200 XP	Le max possible est 2500 XP. Il peut avoir zappé 3 exos cumulés au total.
Expert	35 à 42	2 900 XP	Le max possible est 3400 XP. Il a pu zapper 5 exos depuis le début, il passe quand même !
Maître	43 à 50	4 200 XP	Aucune tolérance. 4200 XP correspond exactement à 42 exos réussis. S'il en a zappé avant, il doit retourner les faire.

5. Modèle de Données (Base de données)

Pour que tout cela fonctionne, tu auras besoin au minimum de ces 3 tables :

1. **Table `users`** : `id`, `username`, `password_hash`, `xp`, `rank`, `created_at`.
2. **Table `exercises`** : `id`, `title`, `description`, `base_code` (code pré-rempli fourni à l'utilisateur), `test_code` (les tests cachés à fusionner), `xp_reward`, `required_rank`.
3. **Table `user_submissions` (Historique)** : `id`, `user_id`, `exercise_id`, `status` (réussi/échoué), `submitted_code`, `date`.

Je voudrais faire en sorte que certains niveaux rapportent plus ou moins d'xp, et que les niveaux multiples de 10 soient des niveaux "boss" qui résument les précédents. Mais en gros si par exemple l'utilisateur n'a pas fini le niv boss de débutant, il n'ait pas assez de points pour passer avancé mais qu'il ait quand même accès au niveaux intermédiaires, pareil pour normal, pas obligé de finir

son boss pour passer à avancer mais obligé pour avoir accès aux niveaux expert
Dis moi ce que t'en penses.

codexia/

-  **controllers/** (Les chefs d'orchestre : *AuthController.php*, *ExerciseController.php*...)
-  **models/** (Ceux qui parlent à la base de données : *User.php*, *Exercise.php*...)
-  **views/** (L'interface HTML/CSS)
 -  **public/** (accueil, login, register)
 -  **app/** (dashboard, tracks, arène)
-  **public/** (Les assets : CSS, images, fichiers JavaScript de Monaco)
-  **index.php** (Le point d'entrée unique du site)

-- 1. Création de la table TRACK

```
CREATE TABLE track (  
  id_track INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  slug VARCHAR(100) UNIQUE NOT NULL,  
  is_active BOOLEAN DEFAULT TRUE  
) ENGINE=InnoDB;
```

-- 2. Création de la table UTILISATEUR (avec niveaux et XP)

```
CREATE TABLE utilisateur (  
  id_utilisateur INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  bio TEXT,  
  avatar_url TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  role VARCHAR(50) DEFAULT 'student',  
  global_level INT DEFAULT 1,  
  global_xp INT DEFAULT 0  
) ENGINE=InnoDB;
```

-- 3. Création de la table de liaison SUIVRE (Niveau de l'utilisateur dans un Track précis)

```
CREATE TABLE suivre (  
  id_utilisateur INT NOT NULL,  
  id_track INT NOT NULL,  
  grade_actuel VARCHAR(50) DEFAULT 'Débutant',
```

```

    track_xp INT DEFAULT 0,
    PRIMARY KEY (id_utilisateur, id_track),
    CONSTRAINT fk_utilisateur_suivre
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur) ON DELETE
    CASCADE,
    CONSTRAINT fk_track_suivre
        FOREIGN KEY (id_track) REFERENCES track(id_track) ON DELETE CASCADE
) ENGINE=InnoDB;

```

-- 4. Création de la table EXERCICE (avec grades et thèmes)

```

CREATE TABLE exercice (
    id_exercice INT AUTO_INCREMENT PRIMARY KEY,
    id_track INT NOT NULL,
    title VARCHAR(200) NOT NULL,
    slug VARCHAR(200) UNIQUE NOT NULL,
    grade_exercice VARCHAR(50) NOT NULL, -- ex: 'Débutant', 'Intermédiaire'
    theme_chapitre VARCHAR(100) NOT NULL, -- ex: 'La base de python'
    description TEXT,
    instructions TEXT,
    CONSTRAINT fk_track_exercice
        FOREIGN KEY (id_track) REFERENCES track(id_track) ON DELETE CASCADE
) ENGINE=InnoDB;

```

-- 5. Création de la table SOUMISSION

```

CREATE TABLE soumission (
    id_soumission INT AUTO_INCREMENT PRIMARY KEY,
    id_utilisateur INT NOT NULL,
    id_exercice INT NOT NULL,
    code TEXT NOT NULL,
    status_soumission ENUM('pending', 'success', 'failed') DEFAULT 'pending',
    version INT DEFAULT 1,
    test_output TEXT,
    CONSTRAINT fk_utilisateur_soumission
        FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur) ON DELETE
    CASCADE,
    CONSTRAINT fk_exercice_soumission
        FOREIGN KEY (id_exercice) REFERENCES exercice(id_exercice) ON DELETE
    CASCADE
) ENGINE=InnoDB;

```

-- 6. Création de la table de liaison PROGRESSER (avec l'ENUM des 2 états)

```

CREATE TABLE progresser (
    id_utilisateur INT NOT NULL,
    id_exercice INT NOT NULL,
    status_progression ENUM('en_cours', 'termine') DEFAULT 'en_cours',
    completed_at TIMESTAMP NULL DEFAULT NULL,
    PRIMARY KEY (id_utilisateur, id_exercice),
    CONSTRAINT fk_utilisateur_progression

```

```

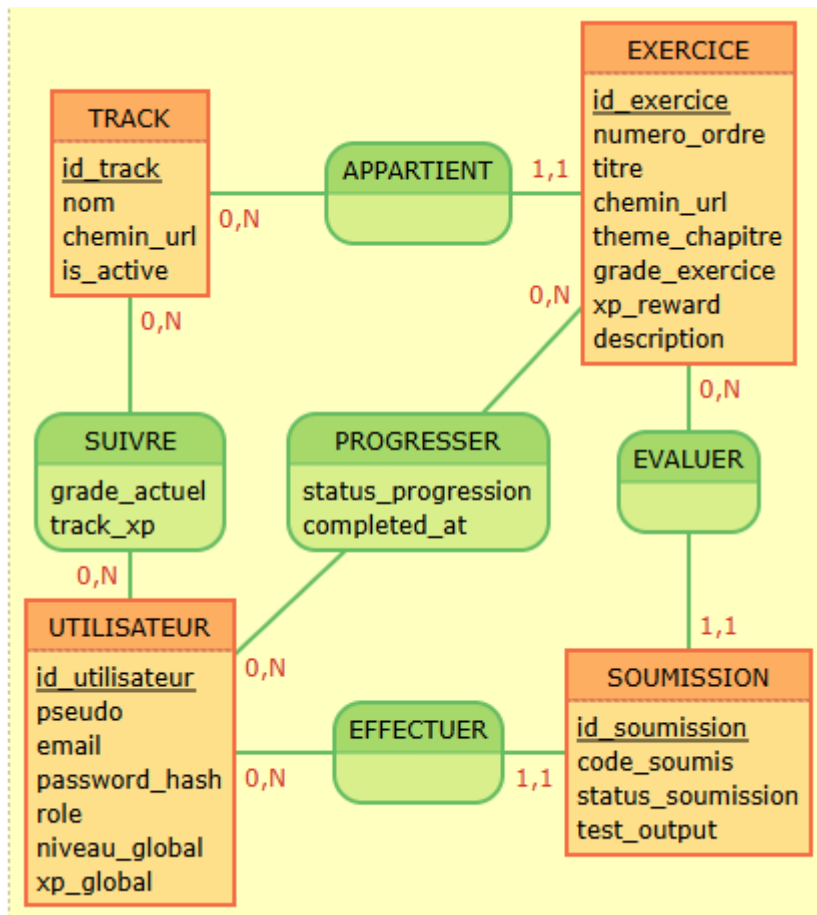
FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur) ON DELETE
CASCADE,
CONSTRAINT fk_exercice_progression
FOREIGN KEY (id_exercice) REFERENCES exercice(id_exercice) ON DELETE
CASCADE
) ENGINE=InnoDB;

```

```

-- Mise à jour de la table EXERCICE
ALTER TABLE exercice
ADD COLUMN xp_reward INT DEFAULT 100,
ADD COLUMN numero_ordre INT NOT NULL DEFAULT 0;
ALTER TABLE exercice
ADD COLUMN base_code TEXT,
ADD COLUMN test_code TEXT NOT NULL;

```



1. La table **TRACK** (Le Parcours d'apprentissage)

Cette table représente un langage de programmation complet (ici, Python).

- **id_track** : L'identifiant unique dans la base (ex: 1).
- **nom** : Le nom affiché du langage (ex: "Python").
- **chemin_url** : L'identifiant propre pour la barre d'adresse (ex: "python").
Votre URL sera `votre-site.com/python`.
- **is_active** : Un simple "Vrai/Faux". Très utile si vous préparez le parcours "JavaScript" en secret : vous le mettez sur "Faux", et les utilisateurs ne le verront pas tant qu'il n'est pas fini.

2. La table **EXERCICE** (Le contenu des cours)

C'est le cœur de votre contenu. Chaque ligne correspondra à l'un de vos 50 exercices.

- **id_exercice** : Identifiant unique (ex: 42).
- **numero_ordre** : Permet de trier l'affichage proprement (ex: 10, 20, 30).
Ainsi, l'exercice "Boucles" s'affichera toujours après "Variables", indépendamment de leur ID.
- **titre** : Le nom de l'exercice (ex: "Découverte des listes").
- **chemin_url** : Pour faire une belle URL (ex: "decouverte-des-listes").
- **theme_chapitre** : Le nom du bloc pour l'affichage (ex: "Conditions & Boucles").
- **grade_exercice** : Le niveau requis pour cet exercice (ex: "Débutant", "Intermédiaire"). Votre code PHP vérifiera ce champ pour savoir s'il doit afficher un cadenas à l'utilisateur.
- **xp_reward** : Le nombre de points que rapporte cet exercice (ex: 100).
- **description** : L'énoncé complet du problème (souvent stocké au format Markdown ou HTML).

3. La table **UTILISATEUR** (Les joueurs / étudiants)

C'est le profil global de la personne inscrite sur votre site.

- **id_utilisateur** : Identifiant unique (ex: 7).
- **pseudo** : Le nom affiché sur le tableau de bord (ex: "Codeur972").
- **email** : Pour la connexion.
- **password_hash** : Le mot de passe (crypté de manière irréversible par PHP).
- **role** : Permet de savoir si c'est un simple étudiant ("student") ou vous-même ("admin") pour vous donner le droit d'ajouter des exercices depuis le site.
- **niveau_global** : Son niveau total sur tout le site (ex: Niveau 5).
- **xp_global** : La somme totale de son expérience.

4. Les tables de liaison (La Gamification et la Progression)

Dans une base relationnelle, quand deux choses sont reliées avec des données supplémentaires (comme le rang ou la date de réussite), cela crée une table dédiée.

La table **SUIVRE** (Profil de l'utilisateur dans un langage précis)

Elle relie un Utilisateur à un Track. C'est ici qu'on gère votre système de "Rangs".

- **grade_actuel** : Le titre honorifique de l'utilisateur pour ce langage (ex: "Intermédiaire", "Maître").
- **track_xp** : L'expérience gagnée *uniquement* sur ce parcours (ex: 1300). C'est ce chiffre que votre PHP regardera pour savoir s'il le fait passer du rang "Débutant" à "Intermédiaire".

La table **PROGRESSER** (L'état d'un exercice)

Elle relie un Utilisateur à un Exercice. C'est votre "Machine à états".

- **status_progression** : L'état actuel de l'exercice (**en_cours** ou **termine**). Si la ligne n'existe pas dans la table, c'est que l'exercice n'a "pas encore été commencé".
- **completed_at** : La date et l'heure exactes de la validation, pour lui afficher un historique ("Exercice réussi le 03 Mars 2026").

5. La table **SOUMISSION** (L'historique du code)

Elle stocke chaque tentative de l'utilisateur. Très pratique si l'utilisateur veut revoir une ancienne solution.

- **id_soumission** : Identifiant unique de la tentative.
- **code_soumis** : Le code Python brut que l'utilisateur a tapé dans l'éditeur Monaco.
- **status_soumission** : Le résultat du test renvoyé par Piston (**pending** quand c'est en cours d'envoi, **success** si les tests passent, **failed** s'il y a une erreur).
- **test_output** : Le message retourné par Piston (ex: "Bravo !" ou l'erreur "SyntaxError: invalid syntax ligne 4"). Cela permet de l'afficher dans la console sur votre site.

merveilleux ! commençons le codage de la page d'accueil, il faudrait :

- le code couleur soit violet et blanc
- une bannière où le nom "Codexia" est écrit

SQL

